Домашнее задание 2. Распределённая файловая система HDFS

Эль-Сибаи Амир, группа ИИИ-3-ИнжДан-1-2

1. Развёртывание

Запустил объектное хранилище RustFS 1.0.1 в Docker 29.8.1 на MacBook (Apple Silicon). Дисками служат четыре папки `disk0`–`disk3` в `~/rustfs-lab`. Каждая подключена в контейнер как отдельный том `/data/rustfs0`–`/data/rustfs3`. Файлы загружал и читал клиентом `mc`.

```
$ mkdir -p ~/rustfs-lab/disk{0..3}
$ docker run -d --name rustfs \
    -p 127.0.0.1:9000:9000 -p 127.0.0.1:9001:9001 \
    -v "$HOME/rustfs-lab/disk0:/data/rustfs0" \
    -v "$HOME/rustfs-lab/disk1:/data/rustfs1" \
    -v "$HOME/rustfs-lab/disk2:/data/rustfs2" \
    -v "$HOME/rustfs-lab/disk3:/data/rustfs3" \
    -e RUSTFS_VOLUMES="/data/rustfs{0...3}" \
    -e RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true \
    -e RUSTFS_ADDRESS=":9000" \
    -e RUSTFS_CONSOLE_ADDRESS=":9001" \
    -e RUSTFS_CONSOLE_ENABLE=true \
    -e RUSTFS_ACCESS_KEY=rustfsadmin \
    -e RUSTFS_SECRET_KEY=rustfsadmin \
    rustfs/rustfs:latest
```

Команду запуска оформил функцией `rustfs_up`, потому что в сценариях отказа она нужна несколько раз.

Без настройки `RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true` хранилище не запускается. Оно проверяет, что диски физически разные, а четыре папки лежат на одном диске ноутбука. Текст ошибки: `local erasure endpoints must use distinct physical disks`. Настройка годится только для локального опыта. В настоящей системе фрагменты обязаны лежать на разных устройствах, иначе поломка одного диска уносит все фрагменты сразу.

Результат: контейнер в статусе `Up`, в журнале видны четыре диска, пустые папки занимают по 44 КБ служебных данных. В `~/rustfs-lab` также лежат папка `source` с исходными данными и папка `draft` с пробным прогоном.

```
[mir@MacBook-Pro ~ % mkdir -p ~/rustfs-lab/disk{0..3} && ls -l ~/rustfs-lab
total 0
drwxr-xr-x  2 mir  staff   64  7 окт.  15:10 disk0
drwxr-xr-x  2 mir  staff   64  7 окт.  15:10 disk1
drwxr-xr-x  2 mir  staff   64  7 окт.  15:10 disk2
drwxr-xr-x  2 mir  staff   64  7 окт.  15:10 disk3
drwxr-xr-x@ 8 mir  staff  256  7 окт.  15:08 draft
drwxr-xr-x@ 5 mir  staff  160  7 окт.  01:13 source
mir@MacBook-Pro ~ % rustfs_up() {
  docker run -d --name rustfs \
    -p 127.0.0.1:9000:9000 -p 127.0.0.1:9001:9001 \
    -v "$HOME/rustfs-lab/disk0:/data/rustfs0" \
    -v "$HOME/rustfs-lab/disk1:/data/rustfs1" \
    -v "$HOME/rustfs-lab/disk2:/data/rustfs2" \
    -v "$HOME/rustfs-lab/disk3:/data/rustfs3" \
    -e RUSTFS_VOLUMES="/data/rustfs{0...3}" \
    -e RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true \
    -e RUSTFS_ADDRESS=":9000" \
    -e RUSTFS_CONSOLE_ADDRESS=":9001" \
    -e RUSTFS_CONSOLE_ENABLE=true \
    -e RUSTFS_ACCESS_KEY=rustfsadmin \
    -e RUSTFS_SECRET_KEY=rustfsadmin \
    rustfs/rustfs:latest
}
[mir@MacBook-Pro ~ % rustfs_up
12bddee31999414715e0f997956078265ee05b3a2fa9b06b53ab1af989c8ffc2
[mir@MacBook-Pro ~ % docker ps --filter name=rustfs; docker logs rustfs 2>&1 | grep -v WARNING; du -sh ~/rustfs-lab/disk*
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS                                      NAMES
12bddee31999   rustfs/rustfs:latest   "/entrypoint.sh rust…"   11 seconds ago   Up 11 seconds   127.0.0.1:9000-9001->9000-9001/tcp   rustfs
Initializing data directories: /data/rustfs0 /data/rustfs1 /data/rustfs2 /data/rustfs3
Initializing log directory: /logs
Starting: /usr/bin/rustfs  /data/rustfs0 /data/rustfs1 /data/rustfs2 /data/rustfs3
 44K    /Users/mir/rustfs-lab/disk0
 44K    /Users/mir/rustfs-lab/disk1
 44K    /Users/mir/rustfs-lab/disk2
 44K    /Users/mir/rustfs-lab/disk3
```

Рис. 1: Запуск RustFS на четырёх папках-дисках

2. Загрузка данных

- Датасет `it_vacancies_2024.csv`, снимок ИТ-вакансий на середину 2024 года: 847 619 384 байта = 808,35 МБ.
- 1000 файлов JSON из архива `hh.zip`, в котором их 10 000. Выбрал случайно, генератор с зерном 42. Вместе 5 525 936 байт = 5,27 МБ, в среднем 5,5 КБ на файл.

Создал три бакета: `vacancies-csv`, `vacancies-json` и `write-test` для проверки записи при отказах.

```
$ mc alias set rustfs http://127.0.0.1:9000 rustfsadmin rustfsadmin
$ mc mb rustfs/vacancies-csv && mc mb rustfs/vacancies-json && mc mb rustfs/write-test
$ mc cp ~/rustfs-lab/source/it_vacancies_2024.csv rustfs/vacancies-csv/
$ mc cp --recursive ~/rustfs-lab/source/hh_sample/ rustfs/vacancies-json/ > /dev/null
$ mc ls --recursive --summarize rustfs/vacancies-json | tail -3
```

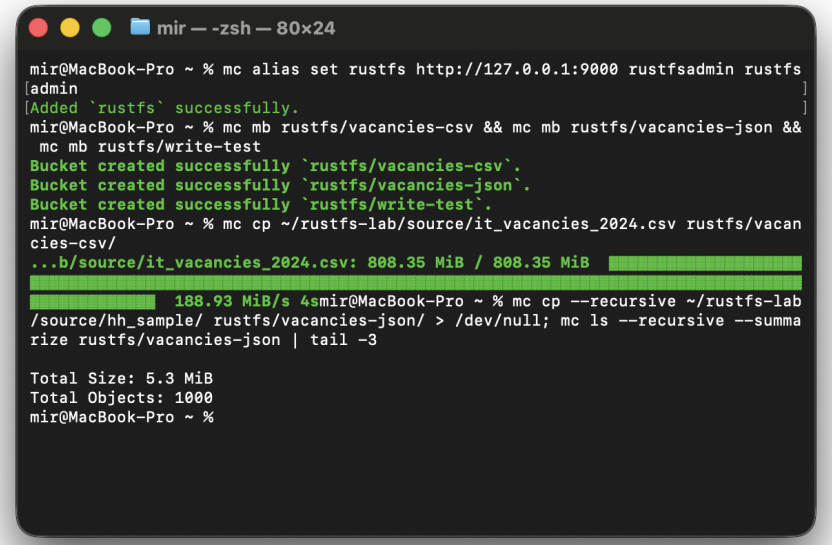


Рис. 2: Создание бакетов и загрузка данных

3. Размеры папок и разделение на фрагменты

Данные	Исходный размер	В каждой папке	Во всех четырёх	Коэффициент
CSV	847 619 384 Б (808,35 МБ)	423 845 223 Б (404,21 МБ)	1 695 380 892 Б	2,0002
1000 JSON	5 525 936 Б (5,27 МБ)	3 342 242 Б (3,19 МБ)	13 368 968 Б	2,42

Все четыре папки совпадают по объёму. Команда `du` показывает 405M для CSV и 4,5M для JSON, потому что считает занятые блоки файловой системы.

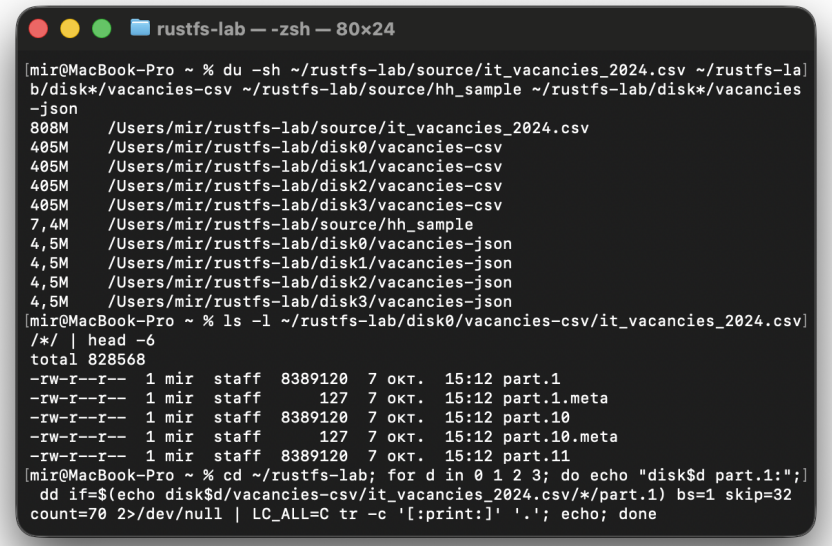


Рис. 3: Размеры папок и файлы фрагментов

Большой файл

- Клиент `mc` загрузил файл 51 частью по 16 МБ. Последняя часть занимает 8 758 584 байта.
- В каждой папке лежит 51 файл `part.N`. Первые 50 занимают по 8 389 120 байт: половина части (8 388 608 байт) + 512 байт контрольных сумм. Последний занимает 4 379 580 байт: половина остатка (4 379 292 байта) + 288 байт.
- $512 = 16 \cdot 32$ и $288 = 9 \cdot 32$. Отсюда вывод: файл режется на блоки по 1 МБ, блок делится пополам на два фрагмента данных, у каждого фрагмента блока есть контрольная сумма в 32 байта.
- Проверка объёма одной папки: $50 \cdot 8\,389\,120 + 4\,379\,580 + 51 \cdot 127 + 3\,166 = 423\,845\,223$ байта. Здесь 127 байт занимает файл `part.N.meta`, 3 166 байт — файл `xl.meta` с описанием объекта.
- По содержимому видно, где данные и где чётность. На `disk0` фрагмент начинается с заголовка CSV: это первая половина блока. На `disk1` лежит текст файла со смещения 524 288 байт: вторая половина блока. На `disk2` и `disk3` байты не читаются: это два фрагмента чётности, вычисленные из двух фрагментов данных.
- Схема: $K = 2$ фрагмента данных + $M = 2$ фрагмента чётности. Коэффициент избыточности $(K + M)/K = (2 + 2)/2 = 2$. Измеренное значение 2,0002, добавку дают контрольные суммы и метаданные.

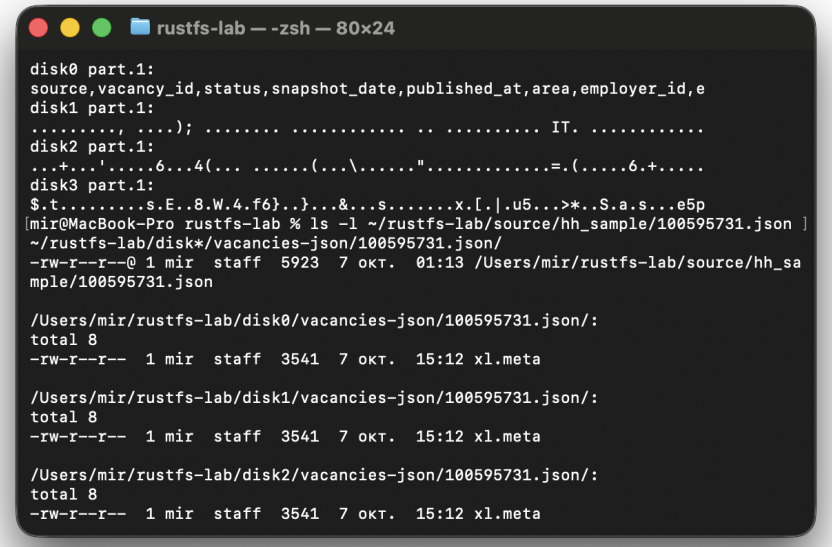


Рис. 4: Начало фрагмента `part.1` в каждой папке

Мелкие файлы

- Отдельных файлов `part.N` нет. Фрагмент лежит внутри файла `xl.meta`, одного на объект в каждой папке.
- Пример: файл `100595731.json` занимает 5 923 байта. В каждой папке его `xl.meta` занимает 3 541 байт: половина файла (2 962 байта) + 579 байт контрольной суммы и метаданных.
- В среднем по 1000 файлов выходит те же 579 служебных байт на объект в каждой папке. Поэтому коэффициент 2,42, а не 2.
- Роли папок меняются от объекта к объекту. Сравнил начало каждого исходного файла и начало его второй половины с содержимым `xl.meta` в каждой папке. На 1000 файлов встретились четыре раскладки (Д1 и Д2 — фрагменты данных, Ч — чётность):

disk0	disk1	disk2	disk3	Объектов
Ч	Ч	Д1	Д2	266
Д2	Ч	Ч	Д1	264
Ч	Д1	Д2	Ч	243
Д1	Д2	Ч	Ч	227

Выделенного диска чётности нет: каждая папка хранит и данные, и чётность.

Контрольная сумма SHA-256 файла, прочитанного из хранилища, совпала с исходной: `303db922...8212a65`. С ней сравниваю чтение в сценариях отказа.

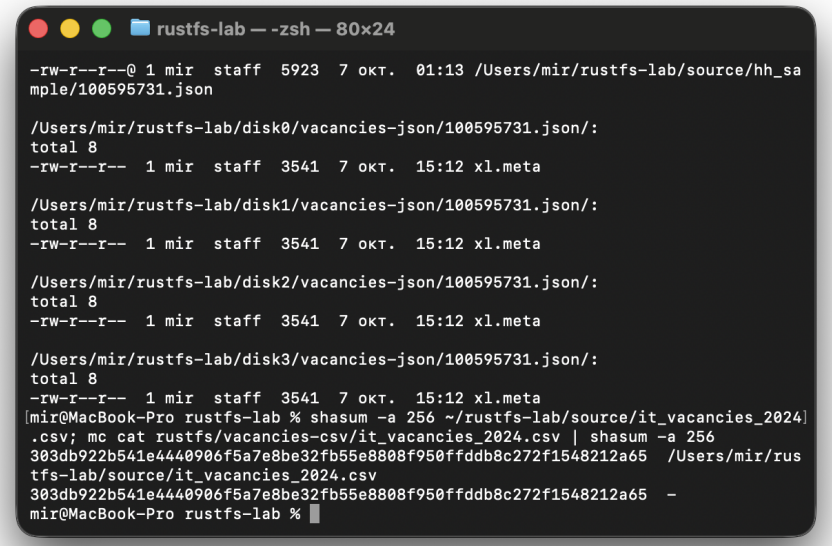


Рис. 5: Файлы `xl.meta` мелкого объекта и сверка контрольных сумм

4. Сценарии отказа

Потерю папки имитировал двумя способами.

- Способ A, папка заменена пустой. Удаляю контейнер, переименовываю папку, запускаю контейнер заново. Docker создаёт на месте пропавшей папки пустую, и хранилище видит чистый диск. Простого перезапуска мало: Docker продолжает отдавать контейнеру переименованную папку.
- Способ B, диск недоступен. Контейнер работает, командой `chmod 000` закрываю доступ к папке, как у сломанного диска.

Способ A. Потеряна 1 папка из 4

```
$ docker rm -f rustfs && mv ~/rustfs-lab/disk3 ~/rustfs-lab/disk3.lost && rustfs_up
$ stat -f '%SB  %N' ~/rustfs-lab/disk*; du -sh ~/rustfs-lab/disk*
$ mc cat rustfs/vacancies-csv/it_vacancies_2024.csv | shasum -a 256; \
    mc cp ~/rustfs-lab/source/hh_sample/100595731.json rustfs/write-test/s1.json
303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65  -
```

- Новая папка `disk3` создана в 15:15:54, остальные в 15:10:29. Через несколько секунд после запуска она уже занимает 406M из 409M. Хранилище само пересчитало потерянные фрагменты по трём оставшимся папкам.
- Чтение CSV: сумма совпала с исходной. Запись нового файла прошла.
- Сравнил восстановленную папку с оригиналом в `disk3.lost`: все 51 фрагмент CSV совпали байт в байт.

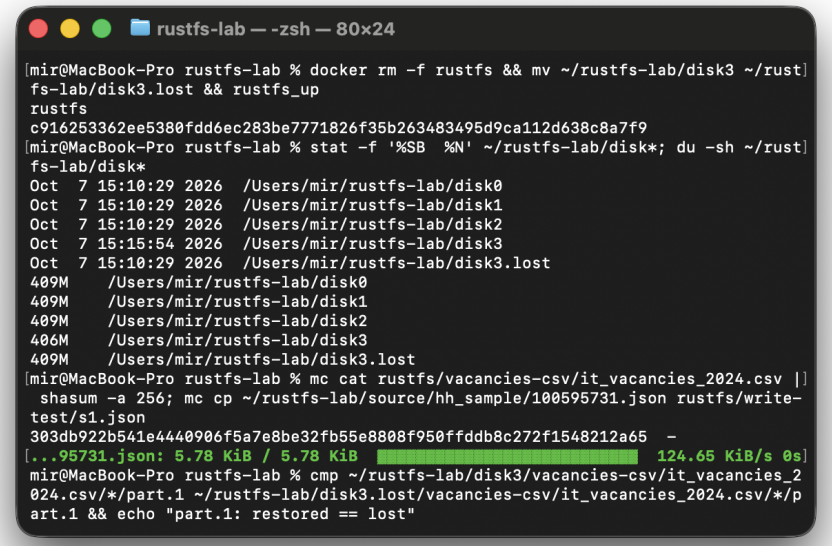


Рис. 6: Способ A, потеряна 1 папка: чтение, запись и самовосстановление

Способ A. Потеряны 2 папки из 4, половина

```
$ docker rm -f rustfs && mv ~/rustfs-lab/disk2 ~/rustfs-lab/disk2.lost && \
    mv ~/rustfs-lab/disk3 ~/rustfs-lab/disk3.healed && rustfs_up
$ du -sh ~/rustfs-lab/disk*; curl -s -w '\n[http %{http_code}]\n' http://127.0.0.1:9000/health/ready
$ mc ls rustfs; mc cat rustfs/vacancies-csv/it_vacancies_2024.csv | shasum -a 256
mc: <ERROR> Unable to list folder. Service not ready: waiting for storage_quorum
mc: <ERROR> Unable to read from `rustfs/vacancies-csv/it_vacancies_2024.csv`.
    Service not ready: waiting for storage_quorum.
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  -
```

- Папки `disk2` и `disk3` пустые, 0 байт. Хранилище не поднялось. Проверка готовности вернула код 503: `ready: false`, `readQuorum: false`, `writeQuorum: false`, причины `storage_and_iam_unavailable` и `startup_finalization_pending`.
- Ничего не прочитано. Сумма `e3b0c442...` — это сумма пустого потока.
- Две папки из четырёх целы, и по математике кода двух фрагментов хватает для восстановления. Но запуститься с половиной пустых дисков хранилище не может. С тремя папками из четырёх оно запускается, с двумя нет.

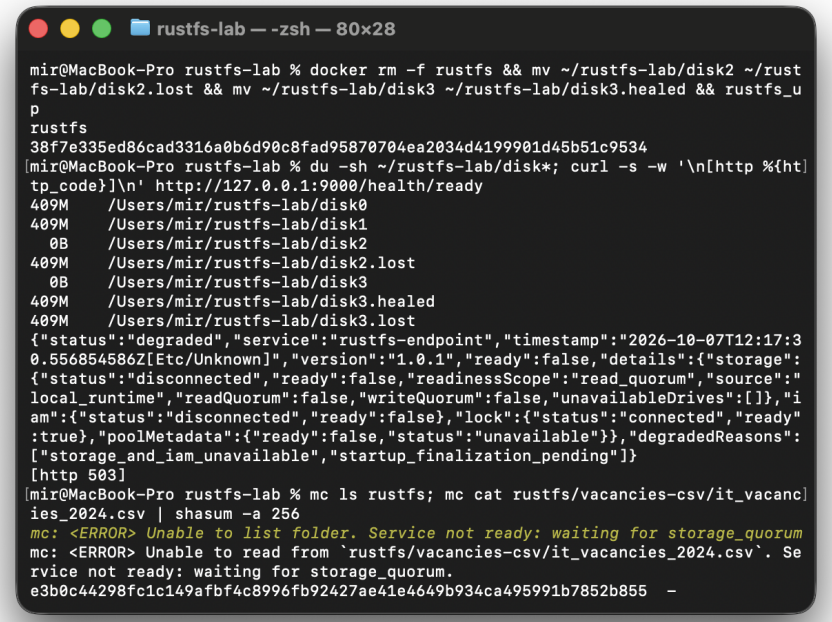


Рис. 7: Способ A, потеряны 2 папки: хранилище не готово

Способ A. Потеряны 3 папки из 4, половина + 1

```
$ docker rm -f rustfs && mv ~/rustfs-lab/disk1 ~/rustfs-lab/disk1.lost && rustfs_up
```

Проверки те же. Результат тот же: код 503, хранилище не готово, данные недоступны. Цела одна папка, одного фрагмента из четырёх для восстановления не хватает.

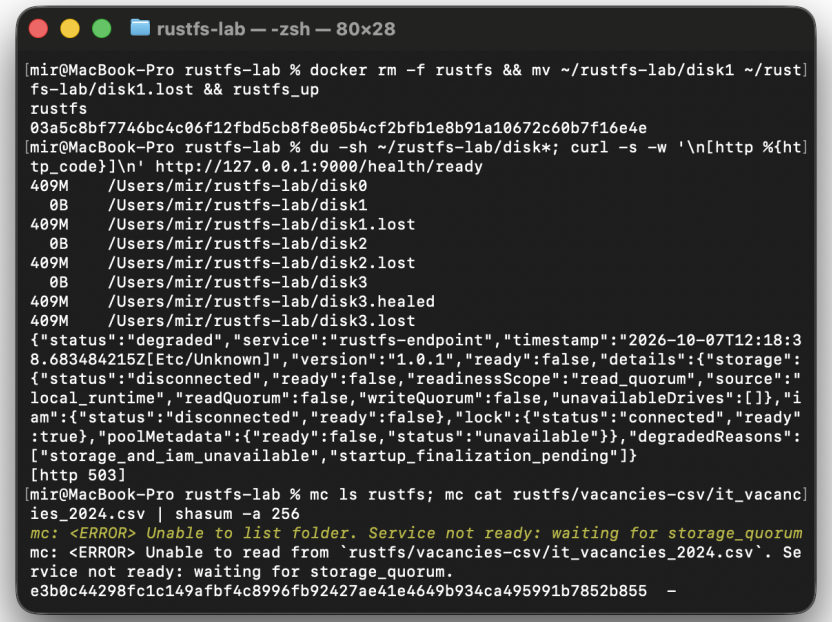


Рис. 8: Способ A, потеряны 3 папки: данные недоступны

После опыта вернул три исходные папки на место. Сумма CSV снова совпала, все папки занимают по 409M.

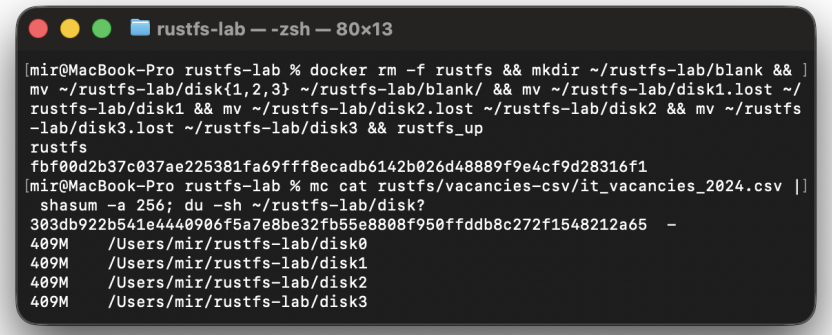


Рис. 9: Возврат исходных папок

Способ B. Доступ закрыт к 1, 2 и 3 папкам

```
$ chmod 000 ~/rustfs-lab/disk3; ls ~/rustfs-lab/disk3
ls: /Users/mir/rustfs-lab/disk3: Permission denied
$ mc cat rustfs/vacancies-csv/it_vacancies_2024.csv | shasum -a 256; \
    mc cp ~/rustfs-lab/source/hh_sample/100595731.json rustfs/write-test/o1.json
303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65  -
...95731.json: 5.78 KiB / 5.78 KiB

$ chmod 000 ~/rustfs-lab/disk1
$ mc cat rustfs/vacancies-csv/it_vacancies_2024.csv | shasum -a 256; \
    mc cp ~/rustfs-lab/source/hh_sample/100595731.json rustfs/write-test/o2.json
303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65  -
mc: <ERROR> Failed to copy ‘/Users/mir/rustfs-lab/source/hh_sample/100595731.json‘.
    The service is unavailable. Please retry.
$ chmod 755 ~/rustfs-lab/disk1 ~/rustfs-lab/disk3

$ chmod 000 ~/rustfs-lab/disk1 ~/rustfs-lab/disk2 ~/rustfs-lab/disk3; ls -ld ~/rustfs-lab/disk?
$ mc cat rustfs/vacancies-csv/it_vacancies_2024.csv | shasum -a 256; mc ls rustfs
mc: <ERROR> Unable to read from ‘rustfs/vacancies-csv/it_vacancies_2024.csv‘.
    Resource requested is unreadable, please reduce your request rate.
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  -
mc: <ERROR> Unable to list folder. The service is unavailable. Please retry.

$ chmod 755 ~/rustfs-lab/disk1 ~/rustfs-lab/disk2 ~/rustfs-lab/disk3
$ mc cat rustfs/vacancies-csv/it_vacancies_2024.csv | shasum -a 256
303db922b541e4440906f5a7e8be32fb55e8808f950ffddb8c272f1548212a65  -
```

- Закрыта 1 папка (`disk3`). Чтение и запись работают.
- Закрыты 2 папки (`disk1` и `disk3`). Файл на 808 МБ читается, сумма совпала. Запись отклонена, список бакетов тоже недоступен.
- Закрыты 3 папки (`disk1`, `disk2`, `disk3`). Чтение даёт ошибку, прочитано 0 байт. Список бакетов недоступен.
- После возврата доступа сумма снова совпала.

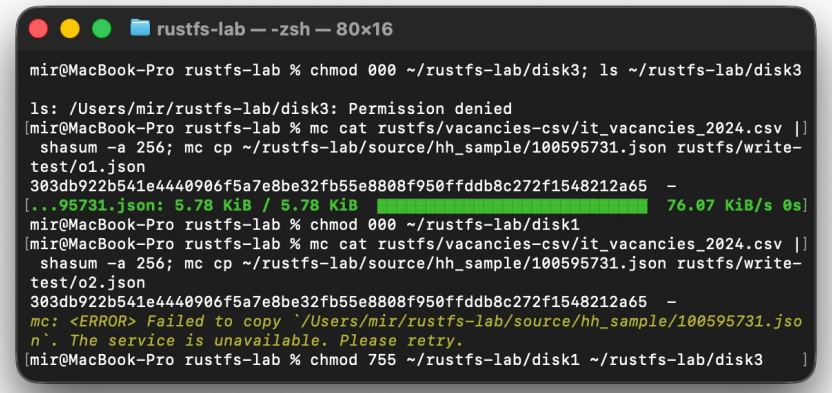


Рис. 10: Способ B, закрыт доступ к одной и к двум папкам

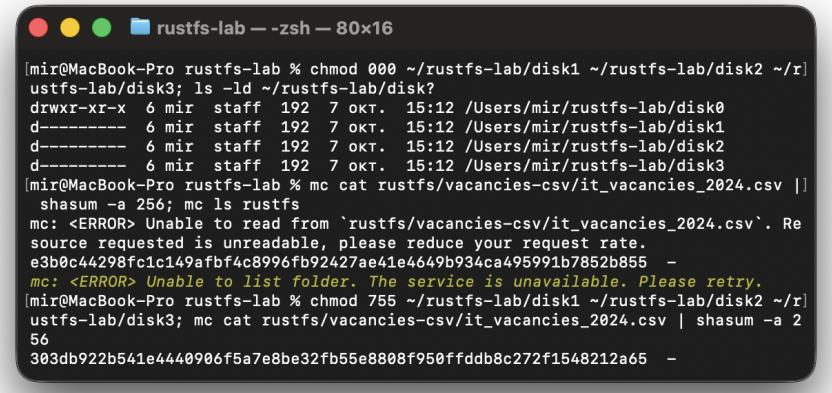


Рис. 11: Способ B, закрыт доступ к трём папкам

Итог по отказам

Потеряно папок	Способ A, папка заменена пустой	Способ B, папка недоступна
1 из 4	Чтение и запись работают, папка восстановилась сама	Чтение и запись работают
2 из 4, половина	Хранилище не запускается	Чтение работает, запись и список бакетов недоступны
3 из 4, половина + 1	Хранилище не запускается	Чтение и список бакетов недоступны, запись не работает уже с двух папок

- Для чтения хватает любых 2 фрагментов из 4. Это число фрагментов данных K.
- Для записи нужны 3 папки из 4, т.е. больше половины. При потере половины хранилище работает только на чтение.
- При потере половины + 1 остаётся 1 фрагмент из 4. Данные недоступны в обоих способах.
- Одну потерянную папку хранилище восстанавливает само на новом пустом диске.
- Холодный запуск строже работы на ходу. С двумя пустыми папками из четырёх хранилище не начинает обслуживать запросы, а на ходу с двумя недоступными папками читает.

5. Расчёт стоимости хранения

Стоимость считаю в сыром объёме дисков: $V_{\text{сыр}} = V \cdot k$, где V — полезный объём, k — коэффициент избыточности.

- Три копии: $k = 3$.
- Erasure Coding с K фрагментами данных и M фрагментами чётности: $k = (K + M)/K$.

Способ хранения	k	Можно потерять	25 ГБ	50 ТБ
Три копии, по умолчанию в HDFS	3	2 копии из 3	75 ГБ	150 ТБ
EC 2 + 2, мой стенд	2	2 диска из 4	50 ГБ	100 ТБ
EC 6 + 3, политика RS-6-3 в HDFS	1,5	3 диска из 9	37,5 ГБ	75 ТБ
EC 12 + 4, 16 дисков в RustFS	1,33	4 диска из 16	33,3 ГБ	66,7 ТБ

- EC 2 + 2 против трёх копий экономит 25 ГБ на малом объёме и 50 ТБ на большом. Это треть сырого объёма.
- EC 12 + 4 против трёх копий на 50 ТБ экономит $150 - 66{,}7 = 83{,}3$ ТБ.
- Поправка по измерениям. На большом файле коэффициент 2,0002, т.е. практически 2. На мелких файлах по 5,5 КБ он равен 2,42. Тогда 25 ГБ займут $25 \cdot 2{,}42 = 60{,}5$ ГБ, а 50 ТБ займут 121 ТБ. Мелкие файлы перед загрузкой стоит объединять в крупные.

6. Обоснование выбора хранилища

25 ГБ: три копии

- EC экономит всего 25 ГБ сырого места. Это малая доля одного диска.
- Три копии переживают потерю двух копий из трёх, для чтения хватает одной целой копии. EC 2 + 2 тоже переживает потерю двух дисков, но на стенде при потере половины дисков перестал принимать запись, а после запуска с половиной пустых дисков не поднялся.
- Трём копиям хватает трёх дисков, EC 2 + 2 нужно минимум четыре. Копии не требуют кодирования и восстановления по чётности.

50 ТБ: Erasure Coding

- EC 2 + 2 экономит 50 ТБ сырого объёма, EC 12 + 4 экономит 83,3 ТБ. На таком объёме экономия измеряется десятками терабайт.
- Схему стоит брать шире, чем 2 + 2. EC 12 + 4 выдерживает потерю 4 дисков из 16 при $k = 1{,}33$. У 2 + 2 запас на запись всего один диск.
- Скорость на стенде не измерял, здесь опираюсь на устройство схемы. При записи считается чётность, при потере диска чтение идёт с восстановлением, а починка читает K фрагментов вместо одной копии. Для архива вакансий, который пишется один раз и читается пакетно, это приемлемо.
- Объекты должны быть крупными. На мелких JSON измеренный коэффициент 2,42 съедает часть выгоды.

7. Выводы

- RustFS на четырёх папках хранит данные по схеме 2 + 2. Измеренный коэффициент избыточности на большом файле 2,0002, на мелких файлах 2,42.
- Потерю одной папки клиент не замечает, хранилище чинит её само. Потеря половины папок оставляет только чтение. Потеря половины + 1 означает, что данные недоступны.
- EC требует в 1,5–2,25 раза меньше сырого объёма, чем три копии (k от 1,33 до 2 против 3), но ему нужно больше дисков, вычисления и крупные объекты.
- Для 25 ГБ выбрал три копии, для 50 ТБ — Erasure Coding со схемой шире, чем 2 + 2.